- **print(object, end='\n', sep=' ')**
- **Escape Charaters:**
  - `\n` **:** New Line
  - `\t` **:** Tab
  - `\\` **:** Backslash
  - `\'` **:** Single Quote
  - `\"` **:** Double Quote
- **Indentation**

In [ ]:
```
print('Hello AMIT')
```

```
Hello AMIT
```

In [ ]:
```
print('Hello', 'AMIT')
```

```
Hello AMIT
```

In [ ]:
```
print('Hello', 'AMIT', sep='-')
```

```
Hello-AMIT
```

In [ ]:
```
print('Hello')
print('AMIT')
```

```
Hello
AMIT
```

In [ ]:
```
print('Hello', end='-')
print('AMIT')
```

```
Hello-AMIT
```

Variables

In [ ]:
```
x = 5
y = 'ali'
z = True
```

Datatypes

- **Text Type:** `str`
- **Numeric Types:** `int`, `float`
- **Boolean Type:** `bool`
- **Sequence Types:** `list`, `tuple`, `set`
- **Mapping Types:** `dict`

`type()`

In [ ]:
```
x = 5
y = 'ali'
z = True

print(type(x))
print(type(y))
print(type(z))
```

```
<class 'int'>
<class 'str'>
<class 'bool'>
```

Strings

- **List Functions & Methods**
  - `len()` : Returns the length of a string.
  - `str()` : Convert an object to a string representation.
  - `sorted()` : Sorts characters in a string and returns it as a new string.
  - `max()` , `min()` : Return the maximum or minimum value from a string.
  - `.lower(), .upper()` : Convert all characters in a string to lowercase or uppercase, respectively.
  - `.capitalize()` : Converts the first character in a string to capital letters.
  - `.title()` : Convert the first character in each word in astring to capital letters.
  - `.find(substring)` : Returns the lowest index in the string where substring is found. If not found, returns -1.
  - `.count()` : Count how many times a substring appears in a string.
  - `.replace('substring', 'substring')` : Replace occurrences of one substring with another.
  - `.format()` , `f"{}"` : Format a string using placeholders and values provided by curly braces.

In [ ]:
```
text = "Hello, World!"
print(len(text))
```

```
13
```

In [ ]:
```
num = 123
string_num = str(num)
print(string_num)
print(type(string_num))
```

```
123
<class 'str'>
```

In [ ]:
```
text = "python"
sorted_text = sorted(text)
print(sorted_text)
```

```
['h', 'n', 'o', 'p', 't', 'y']
```

In [ ]:
```python
text = "abcdef"
print(max(text))
print(min(text))
```

```
f
a
```

In [ ]:
```python
text = "Hello, World!"
print(text.lower())
print(text.upper())
```

```
hello, world!
HELLO, WORLD!
```

In [ ]:
```python
text = "hello, world!"
print(text.capitalize())
```

```
Hello, world!
```

In [ ]:
```python
text = "hello world, welcome to python!"
print(text.title())
```

```
Hello World, Welcome To Python!
```

In [ ]:
```python
text = "hello world"
index = text.find("world")
print(index)

not_found = text.find("python")
print(not_found)
```

```
6
-1
```

In [ ]:
```python
text = "banana"
count = text.count("a")
print(count)
```

```
3
```

In [ ]:
```python
text = "I love apples"
new_text = text.replace("apples", "bananas")
print(new_text)
```

```
I love bananas
```

In [ ]:
```python
name = "Omar"
age = 25
formatted_text = "My name is {} and I am {} years old.".format(name, age)
print(formatted_text)
```

```
My name is Omar and I am 25 years old.
```

In [ ]:
```python
name = "Ali"
age = 30
formatted_text = f"My name is {name} and I am {age} years old."
print(formatted_text)
```

```
My name is Ali and I am 30 years old.
```

Numeric Types

- `int` : integer, positive or negative with unlimited length.
- `float` : floating point number, can represent fractional numbers.

`casting functions`

In [ ]:
```
x = 5
y = -6
z = 3.14
a = -122.53
```

In [ ]:
```
print(type(x))
print(type(y))
print(type(z))
print(type(a))
```

```
<class 'int'>
<class 'int'>
<class 'float'>
<class 'float'>
```

Operators and Expressions

- Assignment operators: `=` , `+=` , `-=` etc.
- Arithmetic operators (`+` , `-` , `*` , `/` , `**` , `//` , `%`) work with numeric types, except `+` it can work with strings as well.
- Comparison operators (`==` , `<` , `<=` , `>` , `>=`) work with any data type that supports them (e.g., text vs numbers).
- Logical operators: `&&` , `||` (logical AND, logical OR).

In [ ]:
```
a = 10
a += 5  # Now a is 15
a *= 2  # Now a is 30
print(a)  # Output: 30
```

```
30
```

In [ ]:
```
x = 15
y = 4

print(x + y)
print(x - y)
print(x * y)
print(x ** y)
```

```
print(x / y)
print(x // y)
print(x % y)
```

```
19
11
60
50625
3.75
3
3
```

In [ ]:
```
a = 10
b = 5

print(a == b)
print(a != b)
print(a > b)
print(a <= b)
```

```
False
True
True
False
```

In [ ]:
```
x = 10
y = 20

print(x > 5 and y < 30)
print(x < 5 or y > 15)
print(not (x == y))
```

```
True
True
True
```

Boolean types: `True`, `False`

In [ ]:
```

```

Sequence Types

`List []`

- A collection of **mutable** items in a particular order.
- Lists are **dynamic** and can grow or shrink as needed.
- Supports efficient insertion/deletion at arbitrary locations.
- Can contain elements of any data type, including custom objects.

- They have **zero-based** index.
- `list()` function is used to create a new list object.

List Functions & Methods

- `len()` : Returns the number of elements in the list.
- `max()` / `min()` : Return the maximum / minimum value from the list.
- `sum()` : Calculate the sum of all values in the list.
- `del()` : Deletes the item at the given index.
- `.append(item)` : Add an element to the end of the list.
- `.extend([items])` : Add multiple items to the end of the list.
- `.insert(pos, item)` : Insert an element at a specific position.
- `.remove(item)` : Remove the first occurrence of a specified value. Raises an error if not found.
- `.pop()` : Remove and return an item at the given position. If no index provided, removes and returns the last item.
- `.clear()` : Removes all items from the list.
- `.copy()` : Returns a copy of the original list.
- `.count` : Count how many times an element appears in the list.
- `.index()` : Find the first occurrence of a value in the list. If not found, raises an error.

In [ ]:
```
names = ['Ahmed', 'ali', 'omar', 'abdu', 'aya', 'mona', 'noha', 'omar']
```

In [ ]:
```
names2 = ['Ahmed', 'ali', 'omar', 'abdu', 'mona', 'noha', 'omar']
```

In [ ]:
```
names.remove('aya')
```

In [ ]:
```
names
```

```
['Ahmed', 'ali', 'omar', 'abdu', 'mona', 'noha', 'omar']
```

In [ ]:
```
names= list([1, 2, 3, 4])
names
```

```
[1, 2, 3, 4]
```

In [ ]:
```
print(len(names))
```

```
4
```

In [ ]:
```
my_list = [5, 15, 25, 10, 30]
print(max(my_list))
print(min(my_list))
```

```
30
5
```

In [ ]:
```
my_list = [1, 2, 3, 4, 5]
print(sum(my_list))
```

```
15
```

In [ ]:
```
my_list = ['a', 'b', 'c', 'd']
del my_list[2]
print(my_list)  # Output: ['a', 'b', 'd']
del(my_list)
print(my_list)
```

```
['a', 'b', 'd']
---------------------------------------------------------------------------

NameError                                 Traceback (most recent call last)

Cell In[35], line 5

      3 print(my_list)  # Output: ['a', 'b', 'd']

      4 del(my_list)

----> 5 print(my_list)



NameError: name 'my_list' is not defined
```

In [ ]:
```
my_list = [1, 2, 3]
my_list.append(4)
print(my_list)
```

```
[1, 2, 3, 4]
```

In [ ]:
```
my_list = [1, 2, 3]
my_list.extend([4, 5, 6])
print(my_list)
```

```
[1, 2, 3, 4, 5, 6]
```

In [ ]:
```
my_list = [1, 2, 4]
my_list.insert(2, 3)
print(my_list)
```

```
[1, 2, 3, 4]
```

In [ ]:
```
my_list = [1, 2, 3, 2]
my_list.remove(2)
print(my_list)
```

```
[1, 3, 2]
```

In [ ]:
```
my_list = ['x', 'y', 'z']
item = my_list.pop()
print(item)
print(my_list)

item = my_list.pop(0)
print(item)
print(my_list)
```

```
z
['x', 'y']
x
['y']
```

In [ ]:
```python
my_list = [1, 2, 3]
my_list.clear()
print(my_list)
```

```
[]
```

In [ ]:
```python
my_list = [1, 2, 3]
my_copy = my_list.copy()
my_copy.append(4)
print(my_list)
print(my_copy)
```

```
[1, 2, 3]
[1, 2, 3, 4]
```

In [ ]:
```python
my_list = [1, 2, 2, 3, 2]
count = my_list.count(2)
print(count)
```

```
3
```

In [ ]:
```python
my_list = ['a', 'b', 'c', 'b']
index = my_list.index('b')
print(index)
```

```
1
```

`Tuple ()`

- Immutable, ordered collection of items.
- Cannot be modified after creation.
- Useful for fixed-size collections where the order of elements matter and you don't want them changed accidentally.
- Tuples can be used to group related data together. For example, **coordinates** (x, y) or **RGB** color values (red, green, blue).
- Can contain elements of any data type, including custom objects.
- They have zero-based.
- `tuple()` function is used to create a new tuble object.

List Functions & Methods

- `len()` : Returns the number of elements in the tuple.
- `max()` / `min()` : Return the maximum / minimum value from the tuple.
- `sum()` : Calculate the sum of all values in the tuple.
- `.count()` : Count how many times an element appears in the tuple.
- `.index(value)` : Find the index position of a specific item within the tuple.

In [ ]:
```
my_tuple = (10, 20, 30, 40, 50)
print(len(my_tuple))
```

```
5
```

In [ ]:
```
my_tuple = (5, 15, 25, 10, 30)
print(max(my_tuple))
print(min(my_tuple))
```

```
30
5
```

In [ ]:
```
my_tuple = (1, 2, 3, 4, 5)
print(sum(my_tuple))
```

```
15
```

In [ ]:
```
my_tuple = (1, 2, 2, 3, 2, 4)
count = my_tuple.count(2)
print(count)
```

```
3
```

In [ ]:
```
my_tuple = ('a', 'b', 'c', 'b')
index = my_tuple.index('b')
print(index)
```

```
1
```

`Set {}`

- A collection of elements.
- The elements in a set are not stored in any particular order.
- Sets cannot contain duplicate elements.
- Can contain elements of any data type, including custom objects.
- They are not subscriptable, meaning you can't access individual elements using square brackets (`[]`).
- `set()` function is used to create a new set object.

List Functions & Methods

- `len()` : Returns the number of elements in the set.
- `max()` / `min()` : Return the maximum / minimum value from the set.
- `sum` : Calculate the sum of all values in the set.
- `del()` : Deletes the set.
- `.add()` :Add an item to the set if not already present.
- `.update()` : Add multiple items to the set.
- `.remove(item)` : Remove a single specified item from the set.
- `.discard()` : Remove a single specified item from the set if it exists.
- `.pop()` : Remove and return an arbitrary item from the set.

- `.clear()` : Removes all items from the set.
- `.copy()` : Create a copy of the original set.
- `issubset()` , `issuperset()` : Check if one set is a subset or superset of another.
- `union()` , `intersection()` , `difference()` : Perform operations on sets.

In [ ]:
```
my_set = {1, 2, 3, 4, 5}
print(len(my_set))
```

```
5
```

In [ ]:
```
my_set = {10, 20, 5, 15}
print(max(my_set))
print(min(my_set))
```

```
20
5
```

In [ ]:
```
my_set = {1, 2, 3, 4, 5}
print(sum(my_set))
```

```
15
```

In [ ]:
```
my_set = {1, 2, 3}
del my_set
print(my_set)
```

```
---------------------------------------------------------------------------

NameError                                 Traceback (most recent call last)

Cell In[53], line 3

      1 my_set = {1, 2, 3}

      2 del my_set

----> 3 print(my_set)



NameError: name 'my_set' is not defined
```

In [ ]:
```
my_set = {1, 2, 3}
my_set.add(4)
print(my_set)
```

```
{1, 2, 3, 4}
```

In [ ]:
```
my_set = {1, 2}
my_set.update([3, 4, 5])
print(my_set)
```

```
{1, 2, 3, 4, 5}
```

In [ ]:
```
my_set = {1, 2, 3}
my_set.remove(2)
print(my_set)
```

```
# my_set.remove(4)  # Raises KeyError: 4 not found in the set
```

```
{1, 3}
```

In [ ]:
```
my_set = {1, 2, 3}
my_set.discard(2)
print(my_set)

my_set.discard(4)  # No error if the item is not found
print(my_set)
```

```
{1, 3}
{1, 3}
```

In [ ]:
```
my_set = {1, 2, 3}
item = my_set.pop()
print(item)
print(my_set)
```

```
1
{2, 3}
```

In [ ]:
```
my_set = {1, 2, 3}
my_set.clear()
print(my_set)
```

```
set()
```

In [ ]:
```
my_set = {1, 2, 3}
my_copy = my_set.copy()
my_copy.add(4)
print(my_set)
print(my_copy)
```

```
{1, 2, 3}
{1, 2, 3, 4}
```

In [ ]:
```
s1 = {12, 22, 5, 16, 5, 7}
s2 = {5, 7, 16, 60, 41, 52, 33}
```

In [ ]:
```
s3 = {12, 22, 5}
```

In [ ]:
```
s3.issubset(s2)
```

```
False
```

In [ ]:
```
s1.issuperset(s3)
```

```
True
```

In [ ]:
```
s1.difference(s2)
```

```
{12, 22}
```

In [ ]:
```
s2.difference(s1)
```

```
{33, 41, 52, 60}
```

In [ ]:
```
s1.union(s2)
```

```
{5, 7, 12, 16, 22, 33, 41, 52, 60}
```

In [ ]:
```
s1.intersection(s2)
```

```
{5, 7, 16}
```

`Dictionary {k:v}`

- A collection of **key-value** pairs.
- The key-value pairs in a dictionary are not stored in any particular order.
- Keys in a dictionary must be **unique**, but values can be duplicated.
- Keys can be of any **immutable** data type, such as integers, strings, or tuples. Values can be of any data type, including custom objects.
- Dictionaries are subscriptable using keys ([]) to access individual values.
- `dict()` function is used to create a new dictionary object.

Dictionary Functions & Methods

- `len()` : Returns the number of key-value pairs in the dictionary.
- `del()` : Deletes the item with the specified key.
- `.keys()` : Returns a view object of all keys in the dictionary.
- `.values()` : Returns a view object of all values in the dictionary.
- `.items()` : Returns a view object of all key-value pairs (tuples) in the dictionary.
- `.get()` : Returns the value of the specified key. If the key does not exist, it returns None (or a default value if provided).
- `.pop()` : Removes and returns the value of the specified key. If the key does not exist, it raises a KeyError (or returns a default value if provided).
- `.popitem()` : Removes and returns an arbitrary key-value pair from the dictionary as a tuple.
- `.clear()` : Removes all key-value pairs from the dictionary.
- `.copy()` : Creates a shallow copy of the dictionary.
- `.update()` : Updates the dictionary with the specified key-value pairs from another dictionary or iterable of key-value pairs.

In [ ]:
```
my_dict = {'a': 1, 'b': 2, 'c': 3}
print(len(my_dict))
```

```
3
```

In [ ]:
```
my_dict = {'a': 1, 'b': 2, 'c': 3}
del my_dict['b']
print(my_dict)
```

```
{'a': 1, 'c': 3}
```

In [ ]:
```
my_dict = {'a': 1, 'b': 2, 'c': 3}
keys = my_dict.keys()
print(keys)
```

```
dict_keys(['a', 'b', 'c'])
```

In [ ]:
```
my_dict = {'a': 1, 'b': 2, 'c': 3}
values = my_dict.values()
print(values)
```

```
dict_values([1, 2, 3])
```

In [ ]:
```
my_dict = {'a': 1, 'b': 2, 'c': 3}
items = my_dict.items()
print(items)
```

```
dict_items([('a', 1), ('b', 2), ('c', 3)])
```

In [ ]:
```
my_dict = {'a': 1, 'b': 2, 'c': 3}
value = my_dict.get('b')
print(value)

# Key not found, returns None
value = my_dict.get('z')
print(value)

# Key not found, returns default value
value = my_dict.get('z', 'Default')
print(value)
```

```
2
None
Default
```

In [ ]:
```
my_dict = {'a': 1, 'b': 2, 'c': 3}
value = my_dict.pop('b')
print(value)
print(my_dict)

# Key not found, returns default value
value = my_dict.pop('z', 'Not Found')
print(value)
```

```
2
{'a': 1, 'c': 3}
Not Found
```

In [ ]:
```
my_dict = {'a': 1, 'b': 2, 'c': 3}
item = my_dict.popitem()
print(item)
print(my_dict)
```

```
('c', 3)
{'a': 1, 'b': 2}
```

In [ ]:
```
my_dict = {'a': 1, 'b': 2, 'c': 3}
my_dict.clear()
print(my_dict)
```

```
{}
```

In [ ]:
```
my_dict = {'a': 1, 'b': 2}
my_copy = my_dict.copy()
my_copy['c'] = 3
print(my_dict)
print(my_copy)
```

```
{'a': 1, 'b': 2}
{'a': 1, 'b': 2, 'c': 3}
```

In [ ]:
```
my_dict = {'a': 1, 'b': 2}
my_dict.update({'b': 3, 'c': 4})
print(my_dict)

# Using an iterable of key-value pairs
my_dict.update([('d', 5), ('e', 6)])
print(my_dict)
```

```
{'a': 1, 'b': 3, 'c': 4}
{'a': 1, 'b': 3, 'c': 4, 'd': 5, 'e': 6}
```

Control Statements

`if..else`

In [ ]:
```
x = 6
y = 6


if x > y:
    print(f"{x} is greater than {y}")

elif y > x:
    print(f"{y} is greater than {x}")

else:
    print(f'{x} is equal to {y}')
```

```
6 is equal to 6
```

In [ ]:
```
x = 6
y = 7
z = 5

if x > y and y > z:
    print(f"{x} is greater than {y} and {y} is greater than {z}")

elif y > x and z > y:
    print(f"{y} is greater than {x} and {z} is greater than {y}")
```

```
elif y > x and y > z:
    print(f"{y} is greater than {x} and {y} is greater than {z}")

elif x > y and z > x:
    print(f"{x} is greater than {y} and {z} is greater than {x}")

else:
    print(f'{x} is equal to {y}')
```

```
7 is greater than 6 and 7 is greater than 5
```

In [ ]:
```
x = input("Enter your name")
print(x)
print(type(x))
```

```
ali
<class 'str'>
```

In [ ]:
```
x = int(input("Enter your Age"))
print(x)
print(type(x))
```

```
25
<class 'int'>
```

In [ ]:
```
x = int(input('Enter X'))
y = int(input("Enter Y"))
z = int(input('Enter Z'))

if x > y and y > z:
    print(f"{x} is greater than {y} and {y} is greater than {z}")

elif y > x and z > y:
    print(f"{y} is greater than {x} and {z} is greater than {y}")

elif y > x and y > z:
    print(f"{y} is greater than {x} and {y} is greater than {z}")

elif x > y and z > x:
    print(f"{x} is greater than {y} and {z} is greater than {x}")

else:
    print(f'{x} is equal to {y}')
```

```
6 is greater than 5 and 7 is greater than 6
```

Loops

- Loops allow you to execute a block of code repeatedly.
- There are two types of loops that we can use inPython:
  - The `for` loop

- The `while` loop

`For`

A for loop is used when you know how many times you want to execute a block of code.

- **Iterating over a `range` of numbers:**

In [ ]:
```
for i in range(5):
    print(i)
```

In [ ]:
```
for i in range(1, 11, 2):
    print(i)
```

In [ ]:
```
for i in range(1, 11):
    if i % 2 == 0:
        print(i)
```

In [ ]:
```
for i in range(1, 11, 2):
    print(i)
```

In [ ]:
```
n = 5
factorial = 1
for i in range(1, n+1):
    factorial *= i
print("Factorial of", n, "is", factorial)
```

In [ ]:
```
n = 5
sum_of_squares = 0
for i in range(1, n+1):
    sum_of_squares += i**2
print("Sum of squares from 1 to", n, "is", sum_of_squares)
```

- **Iterating over characters in a `string`:**

In [ ]:
```
for char in "hello":
    print(char)
```

In [ ]:
```
string = "hello"
vowel_count = 0
for char in string:
    if char in 'aeiou':
        vowel_count += 1
print("Number of vowels:", vowel_count)
```

In [ ]:
```
string = "hello"
reversed_string = ""
for char in string:
```

```
    reversed_string = char + reversed_string
print("Reversed string:", reversed_string)
```

In [ ]:
```
string = "hello"
uppercase_string = ""
for char in string:
    uppercase_string += char.upper()
print("Uppercase string:", uppercase_string)
```

- **Iterating over elements in a `list`:**

In [ ]:
```
my_list = [1, 2, 3, 4, 5]
for num in my_list:
    print(num)
```

In [ ]:
```
my_list = [1, 2, 3, 4, 5]
sum_of_list = 0
for num in my_list:
    sum_of_list += num
print("Sum of list:", sum_of_list)
```

In [ ]:
```
my_list = [1, 2, 3, 3, 4, 4, 5]
seen = set()
duplicates = []
for num in my_list:
    if num in seen:
        duplicates.append(num)
    else:
        seen.add(num)
print("Duplicates:", duplicates)
```

In [ ]:
```
my_list = [4, 7, 2, 9, 1]
max_value = my_list[0]
for num in my_list:
    if num > max_value:
        max_value = num
print("Maximum value:", max_value)
```

- **Iterating over `tuples`:**

In [ ]:
```
my_tuples = [(1, 'a'), (2, 'b'), (3, 'c')]
for tup in my_tuples:
    print(tup)
```

In [ ]:
```
my_tuples = [(1, 2), (3, 4), (5, 6)]
sum_of_pairs = []
for tup in my_tuples:
    sum_of_pairs.append(sum(tup))
print("Sum of pairs:", sum_of_pairs)
```

In [ ]:
```
data = [(1, 'apple', 0.5), (2, 'banana', 0.7), (3, 'orange', 0.6)]
for item in data:
    print("ID:", item[0], "Name:", item[1], "Price:", item[2])
```

In [ ]:
```

```

- **Iterating over key-value pairs in a `dictionary`:**

In [ ]:
```
my_dict = {'a': 1, 'b': 2, 'c': 3}
for key, value in my_dict.items():
    print(key, value)
```

In [ ]:
```
salaries = {'Salaries':{'1': 9500, '2': 12000, '3': 8000, '4': 16500}}
filtered_values = []
for value in salaries.values():
    for salary in value.values():
        if salary > 10000:
            filtered_values.append(salary)
print("Filtered values:", filtered_values)
```

In [ ]:
```
my_dict = {'a': 1, 'b': 2, 'c': 3}
for key in my_dict:
    my_dict[key] *= 2
print("Updated dictionary:", my_dict)
```

`While`

In [ ]:
```
count = 1
while count <= 10:
    print(count)
    count += 1
```

In [ ]:
```
n = 5
factorial = 1
while n > 0:
    factorial *= n
    n -= 1
print("Factorial:", factorial)
```

In [ ]:
```
total = 0
num = 1
while num <= 100:
    total += num
    num += 1
print("Sum:", total)
```

In [ ]:
```
string = "hello"
reversed_string = ""
index = len(string) - 1
while index >= 0:
    reversed_string += string[index]
```

```
    index -= 1
print("Reversed string:", reversed_string)
```

break - continue - pass

In [ ]:
```
numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
for num in numbers:
    if num == 5:
        break
    print(num)
```

In [ ]:
```
while True:
    user_input = input("Enter a word (type 'exit' to quit): ")
    if user_input.lower() == 'exit':
        break
    print("You entered:", user_input)
```

In [ ]:
```
numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
for num in numbers:
    if num % 2 == 0:
        continue
    print(num)
```

In [ ]:
```
my_list = [1, 2, 3, None, 5, None, 7, 8, None, 10]
for item in my_list:
    if item is None:
        continue
    print(item)
```

In [ ]:
```
for i in range(5):
    pass
```